

Semester Planning Agent - Catch-Up Guide for the Team

Team APEX (batch 2_5) - Manhal, Dima, Diyar | July 2026 | course project, due 23/8/2026

What is this project?

A live website where a Technion student plans their next semester with an AI agent that behaves like an experienced academic advisor. The student writes free text (Hebrew or English) - which semester they are starting, what they passed or failed, which days they can't come, how heavy a load they want - and the agent returns a complete plan: the course list with a reason per course, a clash-free weekly timetable (down to lecture/tutorial section choices), an exam calendar with study-day spacing (moed A+B), open risks stated honestly, and a roadmap to graduation. The student can then talk back ('swap this course', 'I can come on Sundays after all') and the agent updates the SAME plan instead of starting over.

Why it is an agent and not a script

There is no hardcoded pipeline. One LLM conversation loop - the **PlanningLoop** module - receives the student's situation plus a toolbox of **15 deterministic Python tools**, and the MODEL decides which tool to call at each step, what to do with the result, and when it is confident enough to call the terminal tool `deliver_plan`. Different student situations genuinely produce different tool sequences (a risk-focused request makes it call risk tools; a tight-days request makes it check the weekly schedule more). Python then re-verifies the delivered plan and strips anything that violates hard rules. Motto of the codebase: **the model is trusted with judgment, never with hard correctness.**

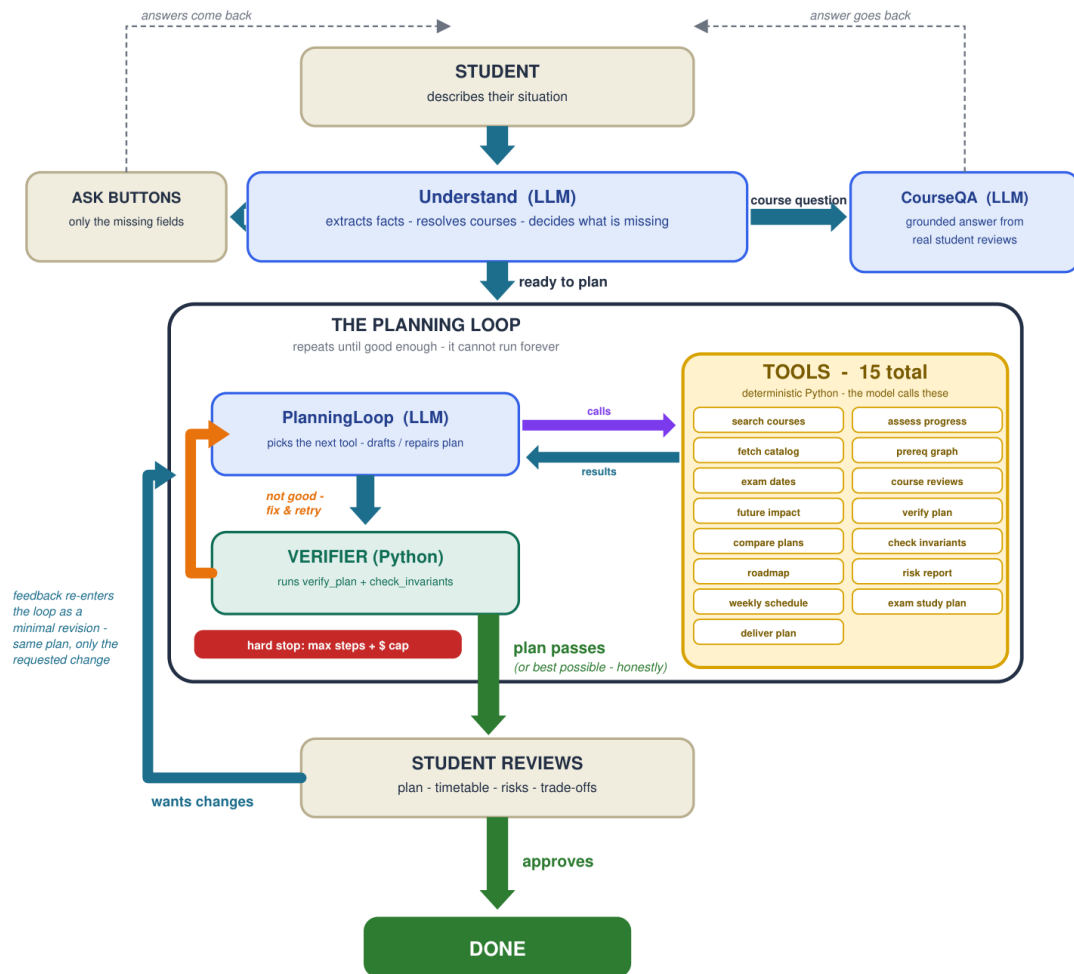
How one request flows (follow the diagram on page 2)

- **1. Understand (LLM module):** reads the whole conversation, extracts structured facts (semester, passed/failed courses resolved to real 8-digit numbers, excluded days, pace, intent). If something essential is missing it asks - but only via buttons for the specific gaps, never re-asking what was already said.
- **2. Branch:** if the message is just a question ('is X hard?') it goes to **CourseQA (LLM module)**, which answers from real student reviews - no plan built. Otherwise: planning.
- **3. PlanningLoop (LLM module):** gets 3 diagnostics pre-injected (progress assessment, degree audit, prereq graph - saves wasted calls) plus the tool menu, and iterates: draft courses, verify, check exams, repair - its own choice of order. Max 18 steps and a per-turn \$ cap; a one-shot guard system pushes back on bad habits (delivering unverified, too many open issues, sport-course padding...).
- **4. deliver_plan (terminal tool):** ends the turn. Python re-runs the verifier and ENFORCES: no schedule collision survives, no already-passed course, no course whose prerequisites aren't met, a requested removal stays removed.
- **5. Student feedback loop:** the reply goes back in as a minimal revision of the same plan (state is carried across turns in `known_context`, merged in code so settled facts are never forgotten).

These module names - Understand, PlanningLoop, CourseQA - are exactly what appears in the steps trace returned by `POST /api/execute`. The professor requires that consistency (diagram = steps = descriptions).

The architecture (served live at GET /api/model_architecture)

Semester Planning Agent - Model Architecture



Two loops drive the agent: the inner plan-verify loop, and the outer student-feedback loop.
The model is trusted with judgment - never with hard correctness.

Blue = LLM modules. They appear under these exact names in the /api/execute steps trace:
Understand - PlanningLoop - CourseQA

Yellow = the 15 TOOLS (deterministic Python) the PlanningLoop model chooses from at every step.
3 diagnostics auto-run each turn (assess progress, fetch catalog, prereq graph); deliver_plan ends the turn.

Blue = the three LLM modules. Yellow = the 15 deterministic Python tools the PlanningLoop model chooses from. Green = deterministic verification and the success path. Red = hard stops. Tan = what the student sees.

The 15 tools (agent/tools.py, schemas in agent/tool_schemas.py)

All tools are pure Python over the pre-fetched data - fast, free, deterministic. 12 are model-choosable; 3 diagnostics (marked *) are auto-run once and injected before the loop starts so the model never wastes calls fetching basics; deliver_plan is terminal (ends the turn).

Tool	What it does
search_courses	find courses by name/topic/number across the track catalog (Hebrew or English)
assess_progress *	compare what the student passed vs. what a typical student at their semester has finished; flags behind/on-track
fetch_catalog *	degree-requirement audit: per category (mandatory, electives...), what is done and how many points remain
query_prereq_graph *	given passed/failed courses: which courses are unlocked now, which are locked and by what
fetch_exam_dates	moed A and moed B dates for any course list
summarize_cheesefork	real student reviews per course: difficulty, workload, tips (the data behind CourseQA too)
simulate_future_impact	look-ahead: taking/skipping course X now - what it unlocks or blocks in future semesters
verify_plan	THE quality gate: prereqs met, credit range for the pace, excluded days respected, section-level schedule collisions, exam spacing - returns a pass/issues report
compare_plans	score two candidate plans side by side (credits, risk, exam crunch, day fit)
check_invariants	sanity backstop: no course both passed and failed, no already-passed course in the plan, retakes marked
roadmap_to_graduation	projected semester-by-semester path to finishing the degree from the current state
risk_report	what could go wrong with a candidate plan: failure-risk courses, heavy combinations, moed-B crunches
weekly_schedule_analyzer	quality of the week itself: density, gaps, long days, best section assignment (interval-partitioning)
exam_study_planner	study days between consecutive exams (A+B); flags 'crunches' (under 3 days) and suggests relief swaps
deliver_plan	terminal: the model hands over course list + per-course why + explanation; Python verifies and enforces before the student sees it

The data: where it comes from and how it is handled

- **Technion SAP portal (public API):** per track - full course catalog (names, points, prerequisites as AND/OR trees, schedules with lecture/tutorial sections, exam dates moed A+B) and the degree requirement tree. Fetched by `pipeline/fetch_track_bundle.py` using `pipeline/technion_api.py`.
- **CheeseFork (community site):** real student reviews per course - difficulty, workload, free-text opinions. Fetched by `pipeline/cheesefork_client.py`.
- **How it is handled:** everything is fetched ONCE offline and saved as one JSON bundle per track - `data/track_SC000001453.json` (Data & Information Eng), `track_SC000001416.json` (Information Systems Eng), `track_SC000001383.json`. At server start `agent/data_bundle.py` loads them and derives structures the tools need: parsed prerequisite trees (`prereq_parser.py`), reverse-prereq graph, and an inferred course ordering (prerequisite depth) since Technion publishes no official 'recommended semester'.
- **Why static JSON and not a live DB:** reproducible, zero external dependency at request time, tool calls are instant and free. Refreshing data = re-running the pipeline scripts and committing the new JSON.

The important code files (read in this order)

File	What it is
<code>agent/agent_react_loop.py</code>	THE AGENT. The tool-calling loop (<code>tool_choice=auto</code>), the guard system, the hard enforcement after <code>deliver_plan</code> , cross-turn revision logic. Start here.
<code>agent/tools.py</code>	the 15 tools (pure Python, all testable without any API key)
<code>agent/tool_schemas.py</code>	OpenAI function-calling schemas for the tools + which are pre-injected
<code>agent/agent_loop.py</code>	the Understand extraction module, state merge across turns, shared helpers, LLM-call trace recording (the steps in <code>/api/execute</code>)
<code>agent/main.py</code>	FastAPI: the 4 course-spec endpoints + <code>/chat</code> + <code>/chat/stream</code> (streaming for the rich UI) + serves the GUI at <code>/</code>
<code>agent/data_bundle.py</code>	loads <code>data/track_*.json</code> at startup, computes prerequisite depths (the inferred semester ordering)
<code>site/index.html</code>	the whole GUI in one file: Run Agent bar (calls <code>/api/execute</code> , shows steps trace), rich chat, timetable grid, exam calendar, diff view, Hebrew RTL, exports
<code>agent/tests/</code>	<code>run_mocked.sh</code> = 54 assertions, zero API cost, run before every push; <code>test_scenarios.py</code> = live tests (costs money)
<code>pipeline/</code>	offline data fetchers (only needed when refreshing data)

Deployment and access (Manhal will share these with you)

- **Runs on Vercel** (serverless): production URL `https://semester-planning-agent-8v6v.vercel.app` - Manhal will share the Vercel project access together with the GitHub repo (`manhal31/semester-planning-agent`, private).
- **Keys:** production uses the course's LLM key + `OPENAI_BASE_URL=https://api.llmod.ai/v1` + model `MB5R2CF-azure/gpt-5.4-mini`, all as Vercel env vars (\$13 group budget). Local dev uses a personal key in gitignored `agent/.env`. NEVER commit a key.
- **Entry point for Vercel:** `api/index.py` + `vercel.json` (300s limit; our runs take 15-60s).
- Docs that exist: **REPORT.pdf** (the full story + features), **TESTING.md** (test plan), **README.md**. This guide is the fast version.

Run it, test it, continue it

Run locally

```
cd plannerProject/agent
python3 -m uvicorn main:app --port 8787
# open http://127.0.0.1:8787/ (needs agent/.env with a key)
bash agent/tests/run_mocked.sh # 54 checks, free - run before every push
```

Check the 4 course-spec endpoints (Project.pdf requires these exact names)

```
B="https://semester-planning-agent-8v6v.vercel.app" # or http://127.0.0.1:8787
curl -s $B/api/team_info | python3 -m json.tool
curl -s $B/api/agent_info | python3 -m json.tool
curl -s $B/api/model_architecture -o arch.png && open arch.png
curl -s -X POST $B/api/execute -H "Content-Type: application/json" \
  -d '{"prompt":"I am starting semester 4 of Data and Information Engineering. I passed everything, no
Sundays, normal pace."}' | python3 -m json.tool
```

Expected from `/api/execute`: `{status:'ok', error:null, response:'...the plan...', steps:[...]}` where every step = `{module, prompt:{System_prompt, User_prompt}, response}` and module is one of Understand / PlanningLoop / CourseQA. All four endpoints were verified working on 22/7 - re-verify on production after every deploy, in an incognito window (no login wall!).

The TODO list - what is left, in order

#	Task	Notes
1	Test the agent much more - be creative, keep an eye on the budget	each <code>/api/execute</code> run costs ~\$0.03-0.06 (LLMod \$13 total). Ideas: all-Hebrew prompts; failed course + blocked days together; 'swap X keep the rest' follow-ups; relaxing a constraint mid-conversation; adversarial 'ignore your rules'; 3 excluded days (impossible squeeze - expect honesty); near-graduation minimum; garbage/empty input; and always check the steps trace matches the diagram. Agree a \$ ceiling BEFORE each batch and stop at it.
2	Decide: should a student be able to give their grade sheet?	not required by Project.pdf. Pasting the transcript TEXT into the prompt already mostly works (Understand resolves course names). Try one cheap test; if good, just mention it in the GUI placeholder. A PDF-upload parser = optional polish, last priority. We only need pass/fail, not grades.
3	Re-check all API endpoints against Project.pdf after every change	names must match EXACTLY (<code>/api/team_info</code> , <code>/api/agent_info</code> , <code>/api/model_architecture</code> , <code>/api/execute</code>); response shapes too (full_response in examples, System_prompt/User_prompt in steps). Commands above.
4	Data source question: is Pinecone mandatory?	the spec lists Supabase (primary DB) + Pinecone (vector DB) but never says 'must'. We run on static JSON and it works. ASK THE PROFESSOR. If required: honest minimal use = embed CheeseFork reviews with the provided embedding model into Pinecone for CourseQA search (~1 day), + log <code>/api/execute</code> runs to Supabase (~half day). Don't fake a DB read just to tick a box.
5	Submit + keep alive	submission format: 'Vercel URL: ... / GitHub Repo URL: ...'. Use the production URL above (NOT a ...-hash.vercel.app link - those are login-walled). Keep the Vercel account active until the grade.

One rule to keep the project healthy: whatever you add, keep every hard guarantee (no collisions, no locked courses, retake included, removal respected) enforced in Python AFTER the model delivers - never trust the model's own bookkeeping.